



Trailhead



Apex Testing (/content/learn/modules/apex\_testing?  
trailmix\_creator\_id=srebello7&trailmix\_slug=salesforce-platform-developer)

>  
Create Test Data for Apex Tests

# Create Test Data for Apex Tests

## Learning Objectives

After completing this unit, you'll be able to:

- Create a test utility class.
- Use a test utility method to set up test data for various test cases.
- Execute all test methods in a class.
- Use test utility classes to add reusable methods for test data setup.

## Prerequisites

Complete the prerequisites in the previous unit, [Test Apex Triggers](https://trailhead.salesforce.com/force_com_dev_beginner/apex_testing/apex_testing_triggers) ([https://trailhead.salesforce.com/force\\_com\\_dev\\_beginner/apex\\_testing/apex\\_testing\\_triggers](https://trailhead.salesforce.com/force_com_dev_beginner/apex_testing/apex_testing_triggers)), if you haven't done so already.

## Add a Test Utility Class

Let's refactor the previous test method by replacing test data creation with a call to a utility class method. First, you need to create the test utility class.

The `TestDataFactory` class is a public class that is annotated with `@IsTest` and can be accessed only from a running test. Test utility classes contain methods that can be called by test methods to perform useful tasks, such as setting up test data. Test utility classes are excluded from the org's code size limit.

To add the `TestDataFactory` class:

1. In the Developer Console, click **File | New | Apex Class**, and enter `TestDataFactory` for the class name, and then click **OK**.
2. Replace the default class body with the following.

```
1  @IsTest
2  public with sharing class TestDataFactory {
3      public static List<Account> createAccountsWithOpps (Integer
4      numAccts, Integer numOppsPerAcct) {
5
```





```

11     insert as user accts;
12     List<Opportunity> opps = new List<Opportunity>();
13     for (Integer j=0; j<numOppsPerAcct; j++) {
14         opps.add(new Opportunity(Name=acct.Name + ' Opportunity
15         // For each account just inserted, add opportunities
16         for (Integer k=0; k<numOppsPerAcct; k++) {
17             opps.add(new Opportunity(Name=acct.Name + ' Opportunity
18             ' + k,
19             StageName='Prospecting',
20             CloseDate=System.today().addMonths(1),

```

Apex Testing (/content/learn/modules/apex\_testing?trailmix\_creator\_id=srebello7&trailmix\_slug=salesforce-platform-developer)

### Create Test Data for Apex Tests

Show More

This test utility class contains one static method, `createAccountsWithOpps()`, which accepts the number of accounts (held in the `numAccts` parameter) and the number of related opportunities to create for each account (held in the `numOppsPerAcct` parameter). The first loop in the method creates the specified number of accounts and stores them in the `accts` list variable. After the first loop, the `insert` DML statement is called to create all accounts in the list in the database.

The second loop creates the opportunities. Because each group of opportunities are linked to one account, the outer loop iterates through accounts and contains a nested loop that creates related opportunities for the current account. The next time the nested loop is run, opportunities are added to the same list using the `add()` method. Opportunities are linked to their parent accounts using the `AccountId` field. The total number of all opportunities that are created is the product of the number of opportunities with the number of accounts (`numOppsPerAcct * numAccts`). Next, the `insert` DML statement is efficiently called outside the loop to create all opportunities in the collection for all accounts in one call only.

Finally, this method returns a list of the new accounts.



Even though this method doesn't return the related opportunities, you can get those records by writing a SOQL query that makes use of the existing relationship between Account and Opportunity, such as the query used in the trigger in [Testing Apex Triggers](https://developer.salesforce.com/en/trailhead/force_com_dev_beginner/apex_testing_data?trailmix_creator_id=srebello7&trailmix_slug=salesforce-pl...) ([https://developer.salesforce.com/en/trailhead/force\\_com\\_dev\\_beginner/apex\\_testing\\_data?trailmix\\_creator\\_id=srebello7&trailmix\\_slug=salesforce-pl...](https://developer.salesforce.com/en/trailhead/force_com_dev_beginner/apex_testing_data?trailmix_creator_id=srebello7&trailmix_slug=salesforce-pl...)).



Here's the modified test method.

[Apex Testing \(/content/learn/modules/apex\\_testing?trailmix\\_creator\\_id=srebello7&trailmix\\_slug=salesforce-platform-developer\)](https://trailhead.salesforce.com/content/learn/modules/apex_testing/apex_testing_data?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)



### Create Test Data for Apex Tests

```

2 private with sharing class TestAccountDeletion {
3     @IsTest
4     static void TestDeleteAccountWithOneOpportunity() {
5         // Test data setup
6         // Create one account with one opportunity by calling a utility
7         method
8         Account[] accts = TestDataFactory.createAccountsWithOpps(1,1);
9         // Perform test
10        Test.startTest();
11        Database.DeleteResult result = Database.delete(accts[0],
12        false, AccessLevel.USER_MODE);
13        Test.stopTest();
14        // Verify that the deletion should have been stopped by the
15        trigger,
16        // so check that we got back an error.
17        Assert.assertFalse(result.isSuccess());
18        Assert.assertTrue(result.getErrors().size() > 0);
19        Assert.areEqual('Cannot delete account with related
        opportunities.',
            result.getErrors()[0].getMessage());
    }

```

## Test for Different Conditions

One test method is not enough to test all the possible inputs for the trigger. We need to test some other conditions, such as when an account without opportunities is deleted. We also need to test the same scenarios with a bulk number of records instead of just a single record. Here is an updated version of the test class that contains the three additional test methods. Save this updated version of the class.

```

1 @IsTest
2 private with sharing class TestAccountDeletion {
3     @IsTest
4     static void TestDeleteAccountWithOneOpportunity() {
5         // Test data setup
6         // Create one account with one opportunity by calling a utility
7         method
8         Account[] accts = TestDataFactory.createAccountsWithOpps(1,1);
9

```





```

14 // Verify that the deletion should have been stopped by the
15 trigger,
16 // and check that we got back an error.
17 trailmix_creator__id=srebello7&trailmix_slug=salesforce-platform-developer)
18 Assert.isTrue(result.getErrors().size() > 0);
19 Create Test Data for Apex Tests
20 Assert.isTrue('Cannot delete account with related
  opportunities '

```

Show More

## Run All Test Methods

The final step is to run the test methods in our test class, now that the class contains more comprehensive tests and has been refactored to use a test data factory. Since you've already run the tests in the `TestAccountDeletion` class, you can just rerun this test class to run all its test methods.

1. To execute the same test run, click the **Tests** tab, select your test run, and then click **Test | Rerun**.
2. Check the results in the Tests tab by expanding the latest test run. The test run should report that all four tests passed!

## Resources

- [Apex Developer Guide: Common Test Utility Classes for Test Data Creation](https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_testing_utility_classes.htm) ([https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex\\_testing\\_utility\\_classes.htm](https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_testing_utility_classes.htm))

### GET READY

You'll be completing this unit in your own hands-on org. Click **Launch** to get started, or click the name of your org to choose a different one.

### YOUR CHALLENGE

Create a Contact Test Factory



NOTE: For the purposes of verifying this hands-on challenge, don't specify the @isTest annotation for either the class or the method, even though it's usually required.

[Apex Testing \(/content/learn/modules/apex\\_testing?trailmix\\_creator\\_id=srebello7&trailmix\\_slug=salesforce-platform-developer\)](/content/learn/modules/apex_testing?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

### Create Test Data for Apex Tests

- Create an Apex class in the `public` scope
  - Name: `RandomContactFactory` (without the @isTest annotation)
  - Use a Public Static Method to consistently generate contacts with unique first names based on the iterated number in the format Test 0, Test 1, and so on
  - Method Name: `generateRandomContacts` (without the @isTest annotation)
  - Parameter 1: An integer that controls the number of contacts being generated with unique first names, such as Test 0 and Test 1
  - Parameter 2: A string containing the common last name of the contacts
  - Return Type: `List<Contact>`

Choose hands-on org

**Koushik nelluri Salesforce CRM**

Created on 5/9/2026



**Launch**

## Check Challenge to Earn 500 Points

### Learn

Badges

Trails

Trailmixes

Superbadges

Trailhead Academy

Career Paths

### Certifications

Certifications

Maintain Certifications

Verify Certifications

### Community

Trailblazer Community

Events

Agentblazers

Salesforce Developers

Salesforce Admins

Code of Conduct

Report Illegal Content (EU)

### Extras

Trailhead Login

Sales Enablement

Trailblazer Store

Salesforce Help



[Apex Testing \(/content/learn/modules/apex\\_testing?trailmix\\_creator\\_id=srebello7&trailmix\\_slug=salesforce-platform-developer\)](#)

[Privacy Statement](#) [Terms of Use](#) [Use of Cookies](#) [Trust](#) [Accessibility](#) [Cookie Preferences](#)



English



[Create Test Data for Apex Tests](#)



[Click for Privacy Choices](#)